| | | |
|---|---|---|
| นางสาวกนกพร บุญครอง | 673380024-0 | Network Architects |
| นางสาวชิดชนก ชนะพา | 673380033-9 | Network Architects |
| นางสาวญาทิชา จันทรศรีสุริยวงศ์ | 673380034-7 | IaC/DevOps |
| นางสาวอรปรีญา แซ่โซ้ง | 673380070-3 | QA/Test |

**Lab 4** using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

# 🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

## 🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

## 👥 TEAM BREAKDOWN (5–6 students)

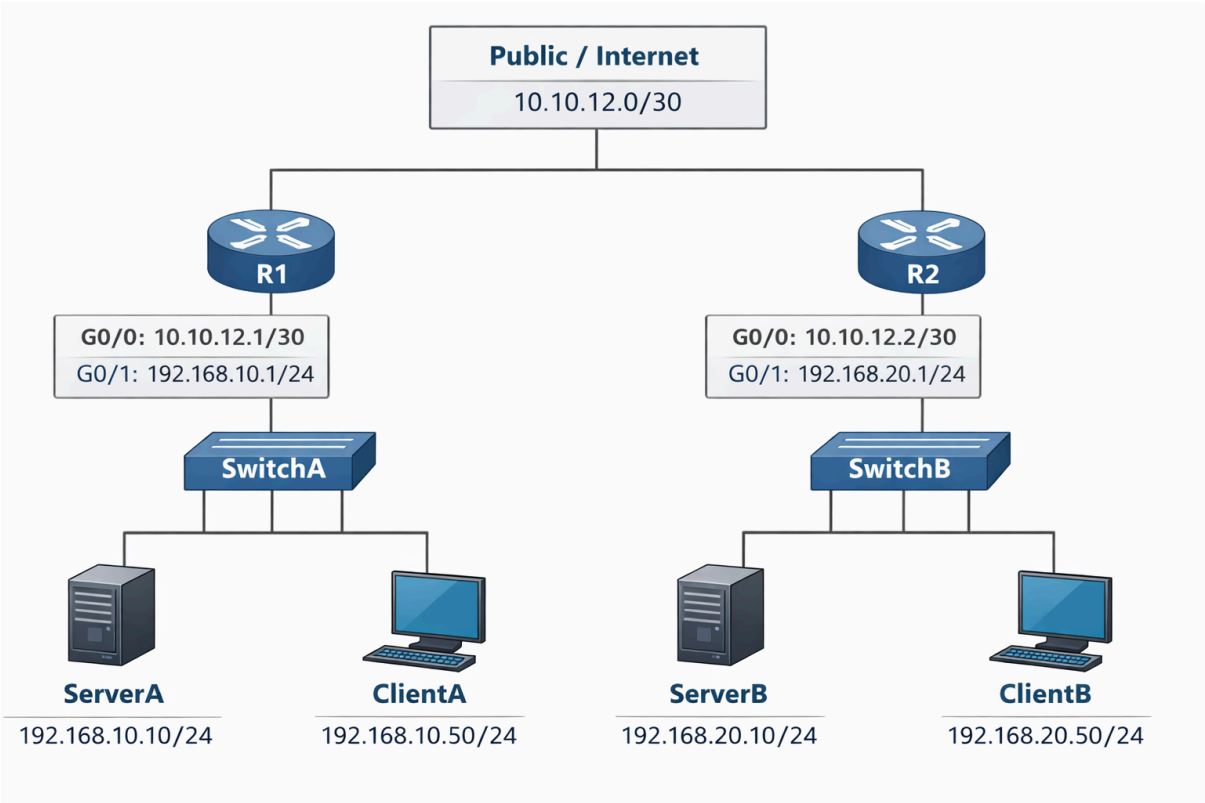
| Role | Count | Focus |
|---|---|---|
| Network Architects | 2 | Router configs, Cisco CLI |
| IaC/DevOps | 2 | VS Code, mockup-infra |
| QA/Test | 1–2 | Server/Client scripting, session behavior |

---

# 🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 “Public/Internet”
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                      SwitchB
             |                          |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
```

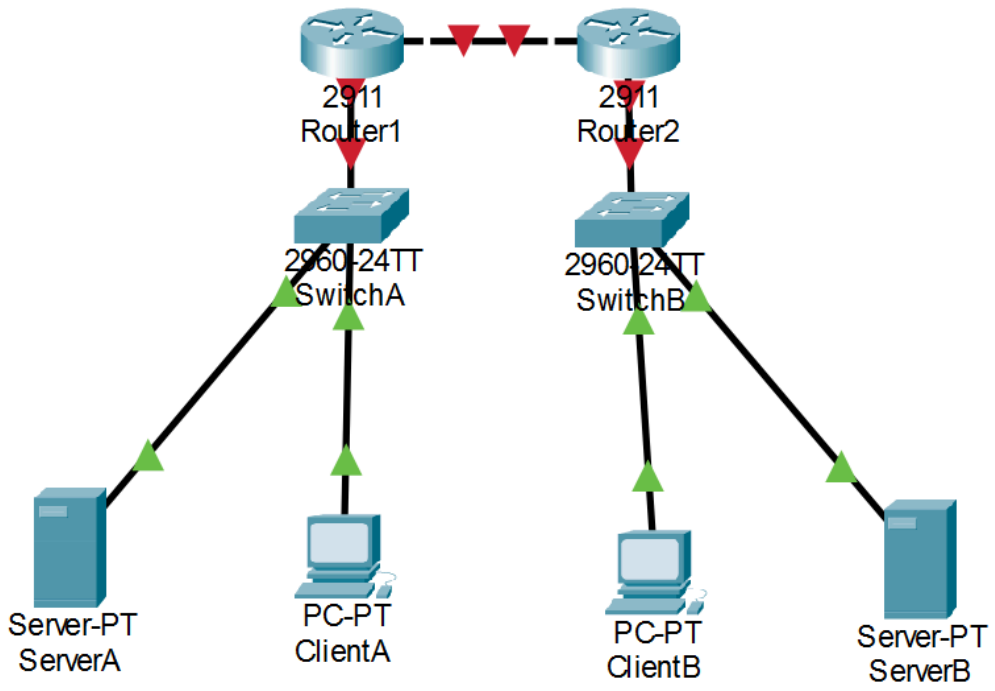
We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.




---

# 📍 ADDRESS TABLE

| Component | Interface | IP | Mask | Role |
|---|---|---|---|---|
| **R1** | G0/0 | 10.10.12.1 | /30 | Public |
| **R1** | G0/1 | 192.168.10.1 | /24 | LAN A Gateway |
| **ServerA** | eth0 | 192.168.10.10 | /24 | Mockup infra A |
| **ClientA** | eth0 | 192.168.10.50 | /24 | Tester |
| **R2** | G0/0 | 10.10.12.2 | /30 | Public |
| **R2** | G0/1 | 192.168.20.1 | /24 | LAN B Gateway |
| **ServerB** | eth0 | 192.168.20.10 | /24 | Mockup infra B |
| **ClientB** | eth0 | 192.168.20.50 | /24 | Tester |




cisco packet tracer simulation
Cisco file :
https://drive.google.com/file/d/1EsBiTN8sdcmOHsSB5M2EztqOZhhwQ9Nh/view?usp=sharing

---

# 🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

---

## 🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1
```

```
interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)# ip address 10.10.12.1 255.255.255.252
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

R1(config-if)#interface g0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat inside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R1(config-if)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0 overload
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

## 🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface g0/0
R2(config-if)# ip address 10.10.12.2 255.255.255.252
R2(config-if)# ip nat outside
R2(config-if)# no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

R2(config-if)#interface g0/1
R2(config-if)# ip address 192.168.20.1 255.255.255.0
R2(config-if)# ip nat inside
R2(config-if)# no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R2(config-if)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0 overload
R2(config)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#write memory
Building configuration...
[OK]
R2#
```

---

## 🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2
```

```
R1#show ip nat translations
Pro  Inside global     Inside local       Outside local      Outside global
icmp 10.10.12.1:1      192.168.10.50:1    192.168.20.10:1    192.168.20.10:1
icmp 10.10.12.1:2      192.168.10.50:2    192.168.20.10:2    192.168.20.10:2
icmp 10.10.12.1:3      192.168.10.50:3    192.168.20.10:3    192.168.20.10:3
icmp 10.10.12.1:4      192.168.10.50:4    192.168.20.10:4    192.168.20.10:4
```

```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/1
```

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

---

# 💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

---

## 🛠️ CLONE REPO

```
Shell
git clone https://github.com/boonsup/automation.git
cd automation
```

---

## 🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

---

## 📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

mockup-infra > ⚙ ServerA.env

```
1  PUBLIC_NET_SUBNET=192.168.10.0/24
2  HOST_IP=192.168.10.10
3  REMOTE_NET_PUBLIC=10.10.12.0/30
```

ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

mockup-infra > ⚙ ServerB.env

```
1  PUBLIC_NET_SUBNET=192.168.20.0/24
2  HOST_IP=192.168.20.10
3  REMOTE_NET_PUBLIC=10.10.12.0/30
```

---

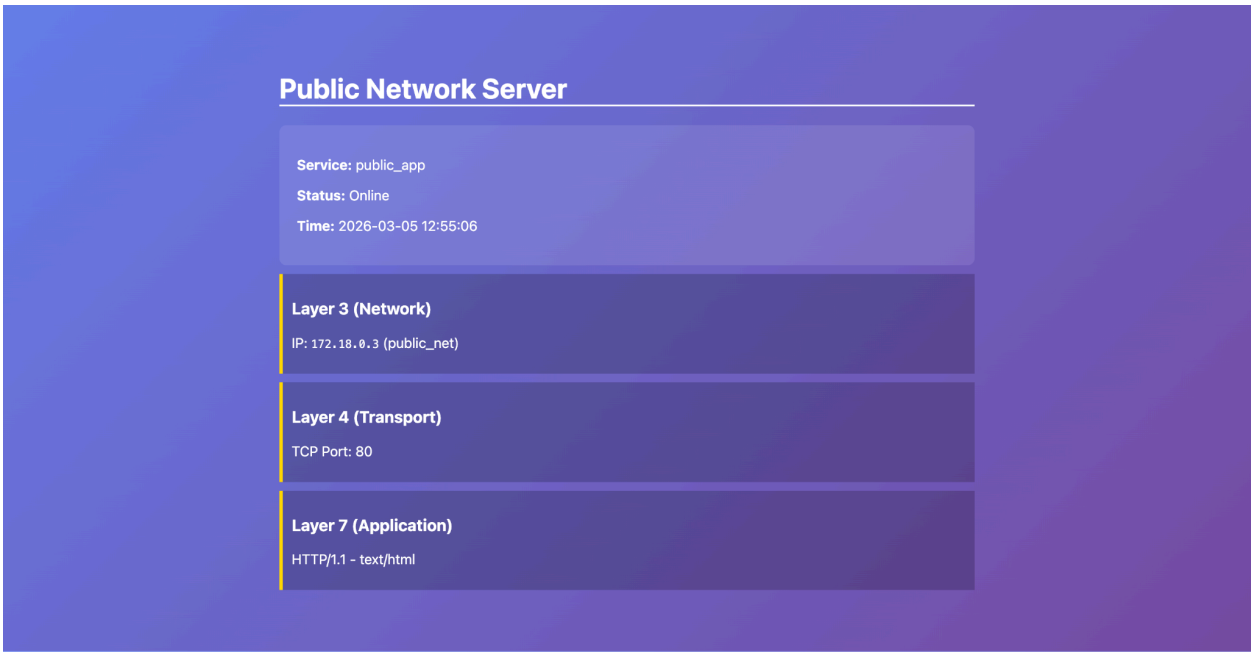
# 🚀 DEPLOY MOCKUP-INFRA

## PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

## Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```



## 🟡 VERIFY CONTAINERS

```
None
podman ps
```

Should show:

| Container | Port |
|---|---|
| stateless-api | 3000 |
| stateful-api | 3001 |

---

# 🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

---

## ⚫ LOCAL TESTS (Private LAN)

From ClientA:

```
Shell
curl http://192.168.10.10:3000/info
```

→ Stateless

```
booboo@Booboos-Macbook-Pro mockup-infra % docker exec stateless-api wget -qO- localhost:3000/info
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"2026-03-05T13:
37:47.981Z","requestCount":1,"clientId":"anonymous","randomValue":0.9226943215015302,"message":"I have no memory of previous requests.
 Each request is independent.","note":"This is a stateless server - I don't remember you from previous requests."},"timestamp":"2026-0
3-05T13:37:47.981Z"}%
```

---

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

การทดสอบในส่วนนี้เป็นการตรวจสอบการทำงานของบริการแบบ **Stateless** และ **Stateful** ภายในเครือข่ายภายใน (Private LAN) โดยทำการส่ง HTTP request จากเครื่อง ClientA ไปยัง ServerA ที่มี IP Address 192.168.10.10

## 1. การทดสอบ Stateless Service

ในการทดสอบบริการแบบ **Stateless Protocol** ได้ใช้คำสั่งดังต่อไปนี้

curl http://192.168.10.10:3000/info

คำสั่งนี้เป็นการส่ง HTTP GET request ไปยัง endpoint `/info` ของเซิร์ฟเวอร์ที่พอร์ต 3000 เพื่อขอข้อมูลเกี่ยวกับสถานะของเซิร์ฟเวอร์ โดยเซิร์ฟเวอร์จะตอบกลับมาในรูปแบบ JSON ที่ประกอบด้วยข้อมูลของระบบ เช่น ชื่อเซิร์ฟเวอร์ เวลาในการตอบสนอง และค่าที่ถูกสุ่มขึ้นมาใหม่ในแต่ละครั้ง

ลักษณะสำคัญของ Stateless คือ เซิร์ฟเวอร์จะไม่เก็บข้อมูลของคำร้องขอก่อนหน้าไว้ ดังนั้นทุก request ที่ส่งเข้ามาจะถูกประมวลผลอย่างอิสระจากกัน ซึ่งเป็นแนวคิดพื้นฐานของโปรโตคอลเว็บอย่าง **HTTP**

---

## 2. การทดสอบ Stateful Service

สำหรับบริการแบบ Stateful จะมีการเก็บสถานะของผู้ใช้งานผ่าน Session โดยเริ่มต้นจากการสร้าง Session ด้วยคำสั่ง

curl -X POST http://192.168.10.10:3001/session

คำสั่งนี้จะส่ง HTTP POST request ไปยัง endpoint `/session` เพื่อให้เซิร์ฟเวอร์สร้าง Session ใหม่สำหรับผู้ใช้งาน หลังจากนั้นเซิร์ฟเวอร์จะตอบกลับมาพร้อม **Session-ID** ซึ่งใช้สำหรับระบุตัวผู้ใช้งานในคำร้องขอถัดไป

จากนั้นจึงใช้ Session-ID ที่ได้รับเพื่อขอข้อมูลจากเซิร์ฟเวอร์ด้วยคำสั่ง

curl http://192.168.10.10:3001/data -H "Session-ID: <id>"

คำสั่งนี้จะส่ง HTTP request พร้อม header ที่มี Session-ID ไปยังเซิร์ฟเวอร์ เพื่อให้เซิร์ฟเวอร์สามารถตรวจสอบและดึงข้อมูลที่เกี่ยวข้องกับ session นั้นได้ การทำงานลักษณะนี้แสดงถึงแนวคิดของ **Stateful Protocol** ซึ่งเซิร์ฟเวอร์จะมีการเก็บข้อมูลสถานะของผู้ใช้งานและสามารถจดจำ session เดิมได้

---

### สรุปผลการทดสอบ

จากการทดสอบพบว่า

- บริการแบบ Stateless จะไม่เก็บข้อมูลของ request ก่อนหน้า ทำให้แต่ละ request ถูกประมวลผลอย่างอิสระ
- บริการแบบ Stateful จะมีการสร้าง Session และใช้ Session-ID เพื่อจดจำสถานะของผู้ใช้งานในการติดต่อครั้งถัดไป

การทดสอบนี้ช่วยแสดงให้เห็นความแตกต่างของการทำงานระหว่าง Stateless และ Stateful services ภายในเครือข่าย Private LAN อย่างชัดเจน

---

## 🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
None
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
None
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

การทดสอบในส่วนนี้เป็นการตรวจสอบการทำงานของระบบเมื่อมีการติดต่อระหว่างเครือข่ายต่างกัน โดยจำลองการเชื่อมต่อผ่านอินเทอร์เน็ต (Simulated Internet) การทดสอบจะส่ง HTTP request จากเครื่อง ClientA ไปยัง ServerB ผ่าน IP สาธารณะของ Router R2 คือ **10.10.12.2** ซึ่ง Router จะทำหน้าที่แปลงที่อยู่ IP ด้วยเทคนิค **Network Address Translation (NAT)** และทำการส่งต่อข้อมูลไปยังเซิร์ฟเวอร์ภายในเครือข่ายปลายทาง

## 1. การทดสอบ Stateless Service ผ่านเครือข่ายภายนอก

ในการทดสอบบริการแบบ **Stateless Protocol** ได้ใช้คำสั่งดังต่อไปนี้

curl http://10.10.12.2:3000/info

คำสั่งนี้เป็นการส่ง HTTP GET request จาก ClientA ไปยัง IP สาธารณะของ Router R2 ผ่านพอร์ต 3000 จากนั้น Router R2 จะทำการแปลงที่อยู่ IP และส่งต่อคำร้องขอไปยัง ServerB ภายในเครือข่าย LAN B

เมื่อ ServerB ได้รับคำร้องขอแล้ว จะตอบกลับข้อมูลของเซิร์ฟเวอร์ในรูปแบบ JSON เช่น ข้อมูลสถานะของระบบ ชื่อเซิร์ฟเวอร์ และค่าที่ถูกสุ่มขึ้นมาใหม่ในแต่ละครั้ง ซึ่งแสดงให้เห็นว่าการทำงานของบริการแบบ Stateless ยังคงทำงานได้ตามปกติแม้ว่าการสื่อสารจะต้องผ่าน Router และ NAT ก็ตาม

---

## 2. การทดสอบ Stateful Service ผ่านเครือข่ายภายนอก

สำหรับบริการแบบ Stateful จะมีการสร้าง session เพื่อเก็บสถานะของผู้ใช้งาน โดยเริ่มต้นจากการส่งคำสั่งเพื่อสร้าง session ดังนี้

curl -X POST http://10.10.12.2:3001/session

คำสั่งนี้จะส่ง HTTP POST request ไปยัง endpoint **/session** ผ่าน IP ของ Router R2 หลังจากนั้น Router จะทำการส่งต่อ request ไปยัง ServerB และเซิร์ฟเวอร์จะสร้าง session ใหม่พร้อมส่ง **Session-ID** กลับมาให้ผู้ใช้งาน

เมื่อได้รับ Session-ID แล้ว จะทำการส่ง request เพื่อขอข้อมูลจากเซิร์ฟเวอร์โดยใช้คำสั่ง

curl http://10.10.12.2:3001/data -H "Session-ID: <id>"

คำสั่งนี้จะส่ง HTTP request พร้อม header ที่มี Session-ID ไปยังเซิร์ฟเวอร์ เพื่อให้เซิร์ฟเวอร์สามารถตรวจสอบ session และดึงข้อมูลที่เกี่ยวข้องกับ session นั้นได้ แม้ว่าการเชื่อมต่อจะผ่านกระบวนการ NAT ระหว่างเครือข่ายก็ตาม

ลักษณะการทำงานนี้แสดงถึงแนวคิดของ **Stateful Protocol** ซึ่งเซิร์ฟเวอร์สามารถจดจำสถานะของผู้ใช้งานผ่าน Session-ID และให้บริการข้อมูลต่อเนื่องได้

---

### สรุปผลการทดสอบ

จากการทดสอบพบว่า

- ClientA สามารถเข้าถึง ServerB ผ่านเครือข่ายจำลองอินเทอร์เน็ตได้สำเร็จผ่านการทำงานของ Routing และ NAT
- บริการแบบ Stateless ยังคงตอบสนองต่อ request ได้ตามปกติ โดยไม่จำเป็นต้องเก็บข้อมูลของ request ก่อนหน้า
- บริการแบบ Stateful สามารถสร้างและรักษา session ของผู้ใช้งานได้ แม้ว่าการเชื่อมต่อจะผ่านกระบวนการ NAT ระหว่างเครือข่าย

ผลการทดสอบนี้แสดงให้เห็นว่าระบบเครือข่ายสามารถรองรับการสื่อสารทั้งแบบ Stateless และ Stateful ได้อย่างถูกต้องแม้จะมีการเชื่อมต่อข้ามเครือข่ายผ่าน Router และ NAT.

---

# ⚠️ TROUBLESHOOTING

## ❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
```

```
ping 10.10.12.2 from R1
```

### ❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

---

## 🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

---

## 📄 LAB README.md (for repository)

None

```
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```

---